

토큰화 개요(5.0): 말뭉치를 토큰 단위로 쪼개어 모델이 처리 가능하게 만들. 공백 분할·정규식·사전 기반·ML 기반 등 방법에 따라 성능이 크게 달라짐.

단어/글자 토큰화(5.1):

- 단어 토큰화는 일반적·간단하지만 OOV와 접사 처리 한계가 있음.
- 글자/자모 토큰화는 어휘를 작게 유지해 OOV를 줄이되, 의미 조합을 모델이 학습해야 하므로 시퀀스 길이·연산량 증가가 단점.

형태소 토큰화(5.2): 한국어 같은 교착어에서 중요. 형태소 사전과 품사 정보를 이용해 단위를 분해/태깅. KoNLPy(Okt, Kkma, Komoran 등)로 명사·구문·품사 태깅 가능하나 분석기마다 결과 차이가 있어 목적에 맞는 선택 필요. NLTK·spaCy는 영어 중심이지만 범용 NLP 기능 제공(spacy는 속도/정확도 지향).

기존 형태소 분석기의 한계: 사전 의존으로 OOV에 취약, 띄어쓰기/품사 오류 전파, 도메인·다국어 확장 유연성 부족.

하위 단어(Subword) 토큰화(5.3): 단어와 문자 사이 '중간 단위'로 분해해 OOV를 큰 폭으로 감소, 어미/접사 변화에 강함. 예: "토큰나이저"→"토큰", "##나이", "##저".

BPE 핵심: 문자에서 시작해 가장 빈번한 인접 쌍을 반복 병합. 병합 규칙(merges)+ 어휘(vocab) 산출, 간단하고 신어 일반화에 강함.

SentencePiece & Korpora 실습:

1. Korpora로 코퍼스 수집→corpus.txt 생성 → 2) SentencePiece BPE 학습(vocab_size 예: 8000) → 3) encode/decode 확인 → 4) 특수 토큰(<unk>, <s>, </s>)와 vocab 크기 확인. 공백 비의존·정규화 내장으로 한국어에 유리.

WordPiece 개념: BERT 계열에서 사용, 빈도만 보지 않고 언어모델의 우도 기준으로 어휘 선택(보통 접두사 ## 사용). 빈도·점수 혼합 휴리스틱 구현이 일반적.

Hugging Face tokenizers 실습: 러스트 기반 고성능. 정규화(NFD/Lowercase), 프리토큰화(Whitespace 등) 설정→WordPiece 학습→JSON으로 저장/로딩→배치 인코딩·디코딩 까지 확인. 한국어는 ByteLevel/Metaspace 옵션도 고려.

토큰나이저 학습 공정 체크리스트: 데이터 수집→정규화 규칙→프리토큰화→모델/어휘 크기 결정→[CLS]/[SEP] 템플릿→평가(토큰 길이·커버리지·OOV·간접 PPL)→버전 고정·배포.

임베딩 개요(6.0): 텍스트를 수치화. BoW/TF(빈도)/TF-IDF는 쉽지만 희소·문맥 미반영. 워드임베딩(Word2Vec/fastText)은 의미 공간을 학습하지만 고정 벡터라 문맥·다의어 한계

→동적 임베딩(언어모델 기반)으로 보완.

언어모델(6.1) & N-gram(6.2): 조건부 확률의 연쇄법칙으로 다음 토큰을 예측(자기회귀). 통계적 LM(N-gram)은 간단하지만 데이터 희소성 문제. GPT/BERT 등은 대규모 코퍼스 기반 확률 모델로 문맥 예측 성능 향상.

TF-IDF(6.3): TF와 IDF로 단어 중요도 가중. TfidfVectorizer 주요 파라미터 (ngram_range, max_df, min_df, stop_words, smooth_idf 등)와 fit/transform/fit_transform 흐름 정리. 한계: 단어 순서·문맥·의미 유사성 미반영.